

Lab 3: Architectural Justification

Scenario: Self-Service Coffee Kiosk System

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Architectural Choice

The kiosk is a single, standalone device (not a distributed, multi-client system and not a service that must scale independently across a network), so a three-tier Layered Architecture — Presentation (Touch Screen UI), Business Logic (Order Manager, Payment Service), and Data/Hardware Interface (Menu Catalog, Receipt Printer Service) — is the best fit. It keeps the design simple to build, test, and maintain within the kiosk's fixed hardware footprint, while still cleanly separating concerns.

Two Reasons for This Choice

1. Predictable, low-latency local flow: Every order at the kiosk follows the same fixed path — UI input → order logic → payment → receipt print — with a single customer per transaction. A layered structure maps directly onto this linear flow without the network hops, service discovery, or distributed data consistency issues that a microservices or client-server split would introduce for a single-device system.

2. Easy maintenance for a fixed feature set: The kiosk's requirements (3 coffee types, 2 sizes, card-only payment, one printer) are stable and well-bounded. Layers let the café's IT team update the menu/pricing data or swap the payment provider inside a single layer without touching the UI or hardware-interface code, which is simpler to operate and support than managing independently deployed microservices for one device.

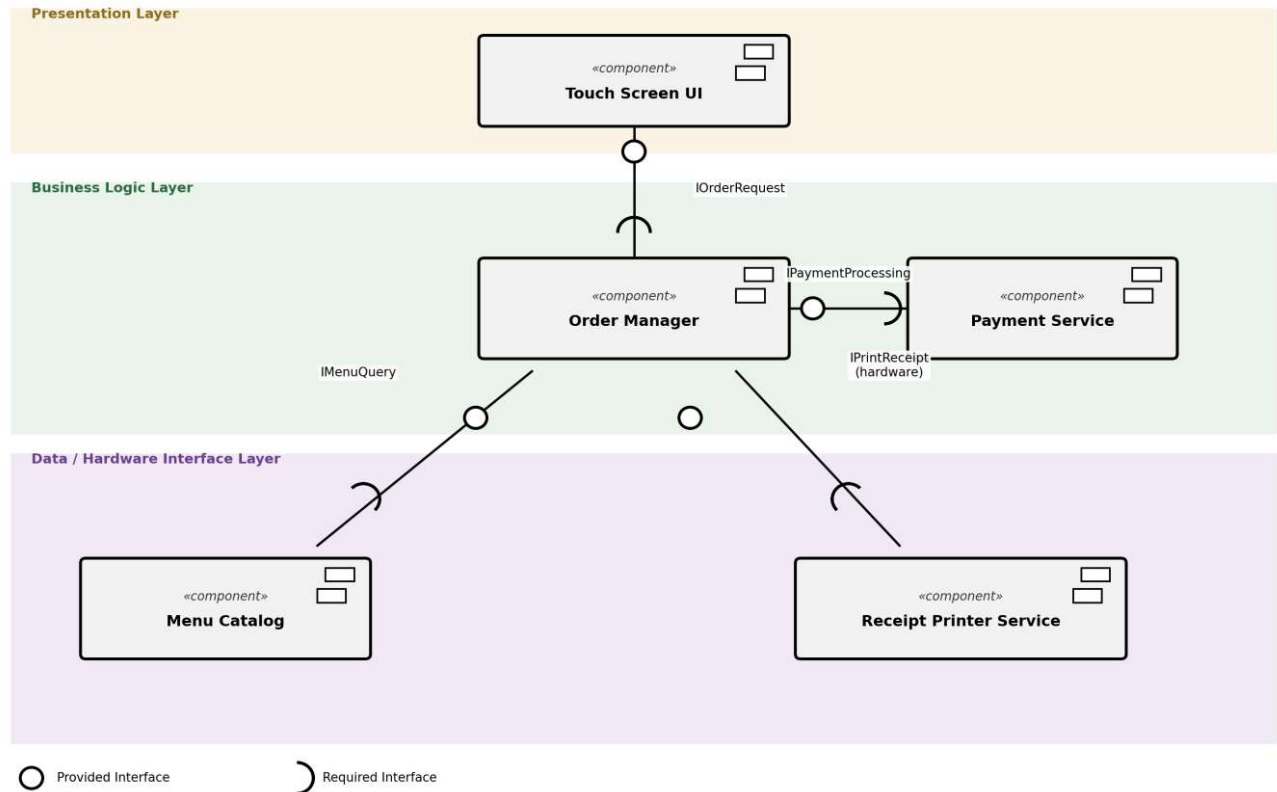
Security Advantage

Isolating the Payment Service in its own business-layer component behind the `IPaymentProcessing` interface means the Touch Screen UI and Menu Catalog never handle raw card data directly — all card input is funneled through one well-defined, auditable boundary. This narrows the attack surface and makes it straightforward to apply PCI-DSS controls (encryption, tokenization) to a single component rather than across the whole system.

Performance Benefit

Because all layers run locally on the kiosk hardware, calls between the Order Manager, Menu Catalog, and Receipt Printer Service are in-process or local hardware calls rather than network requests. This avoids the network latency and round-trip overhead that a microservices or remote client-server design would add, keeping the order-to-receipt experience fast for a customer standing at the kiosk.

UML Component Diagram — Self-Service Coffee Kiosk System (Layered Architecture)



UML Component Diagram — Self-Service Coffee Kiosk System (Layered Architecture)

